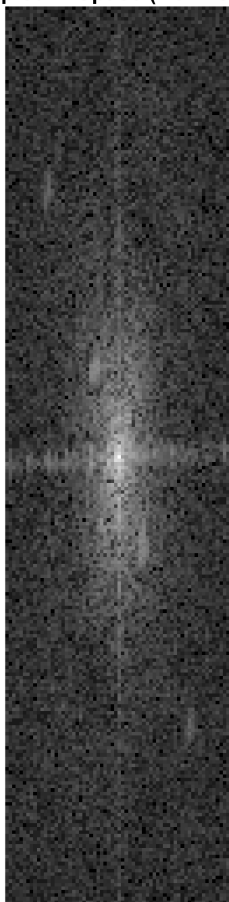


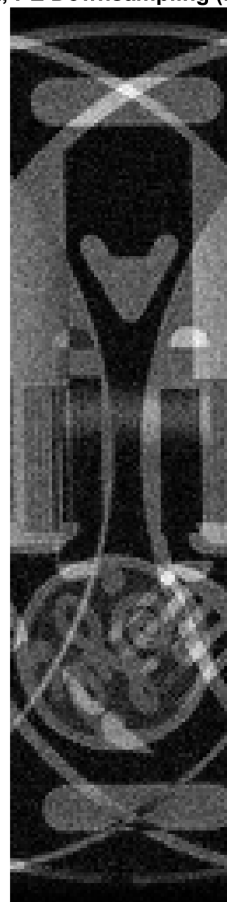
Additional Images for Question 2

Extra Images for Question 2.4

Downsampled K-Space (Downsample = 4) Reconstruction, PE Downsampling (Downsample = 4)



(a) Downsampled K-Space

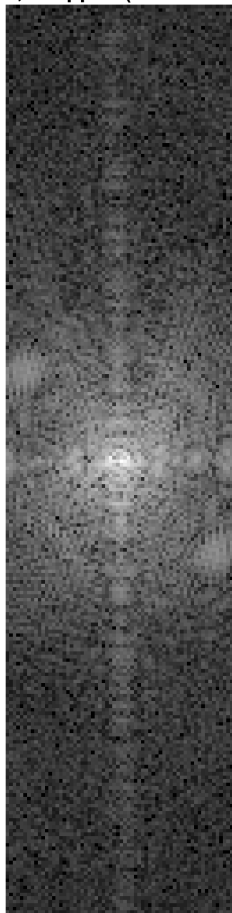


(b) Reconstruction

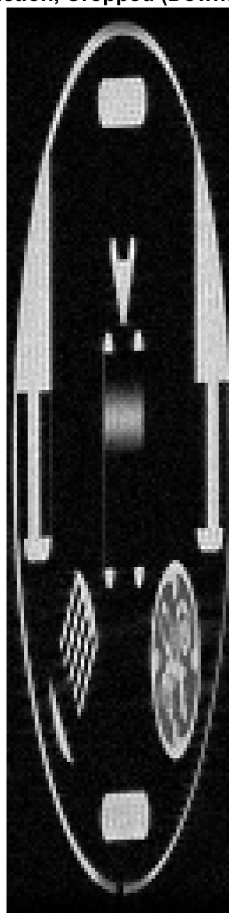
Figure A.1.1: Downsampled Image (By 2)

Extra Images for Question 2.7

K-Space, Cropped (Downsample = 4) Reconstruction, Cropped (Downsample = 4)



(a) Downsampled K-Space



(b) Reconstruction

Additional Images for Question 4

All 5 iterations for questions 4.3, 4.4, 4.5

Extra Images for Question 4.3

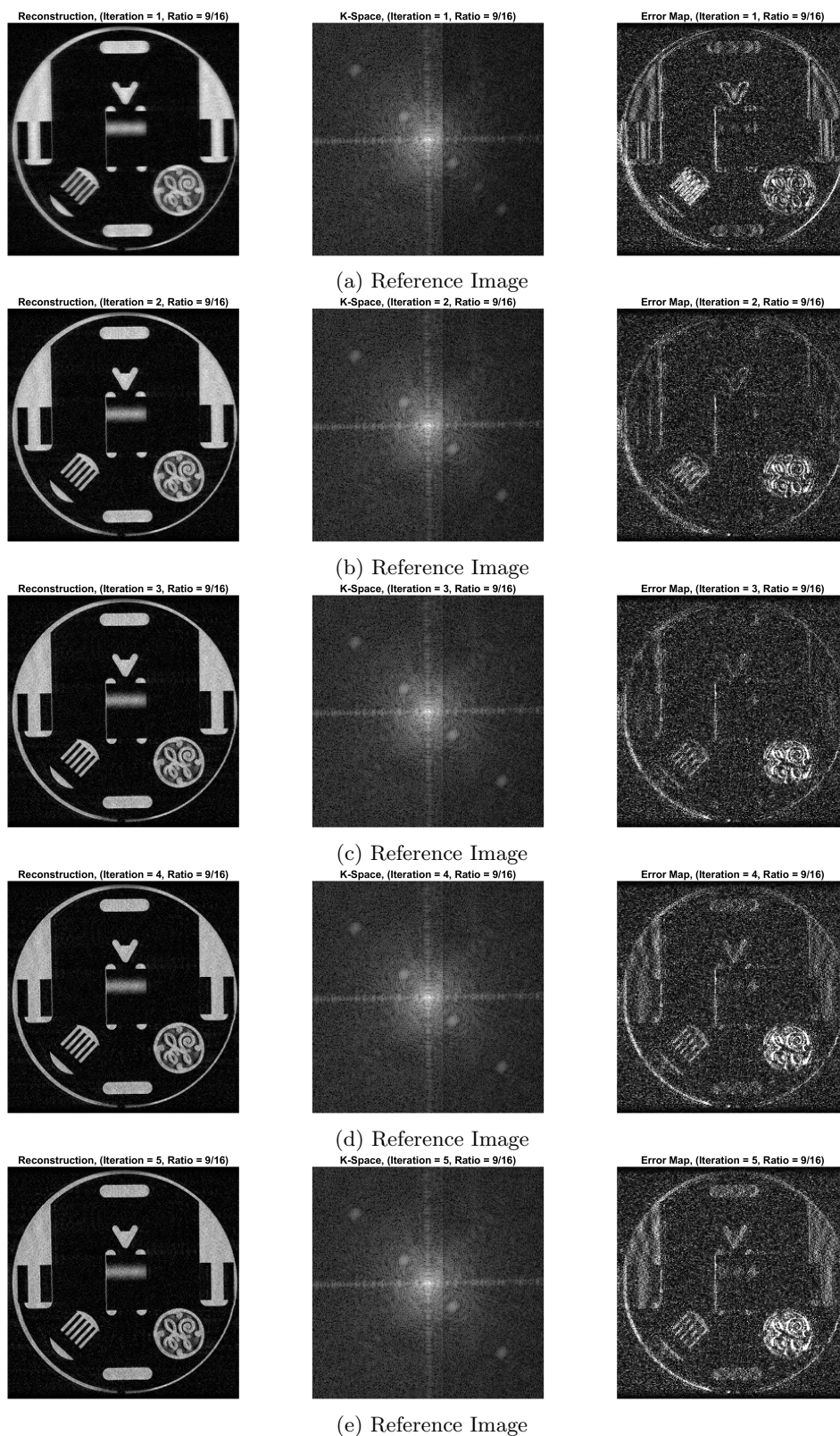


Figure B.1.1: First 5 Iterations of the POCS Algorithm with Ratio = 9/16

Extra Images for Question 4.4

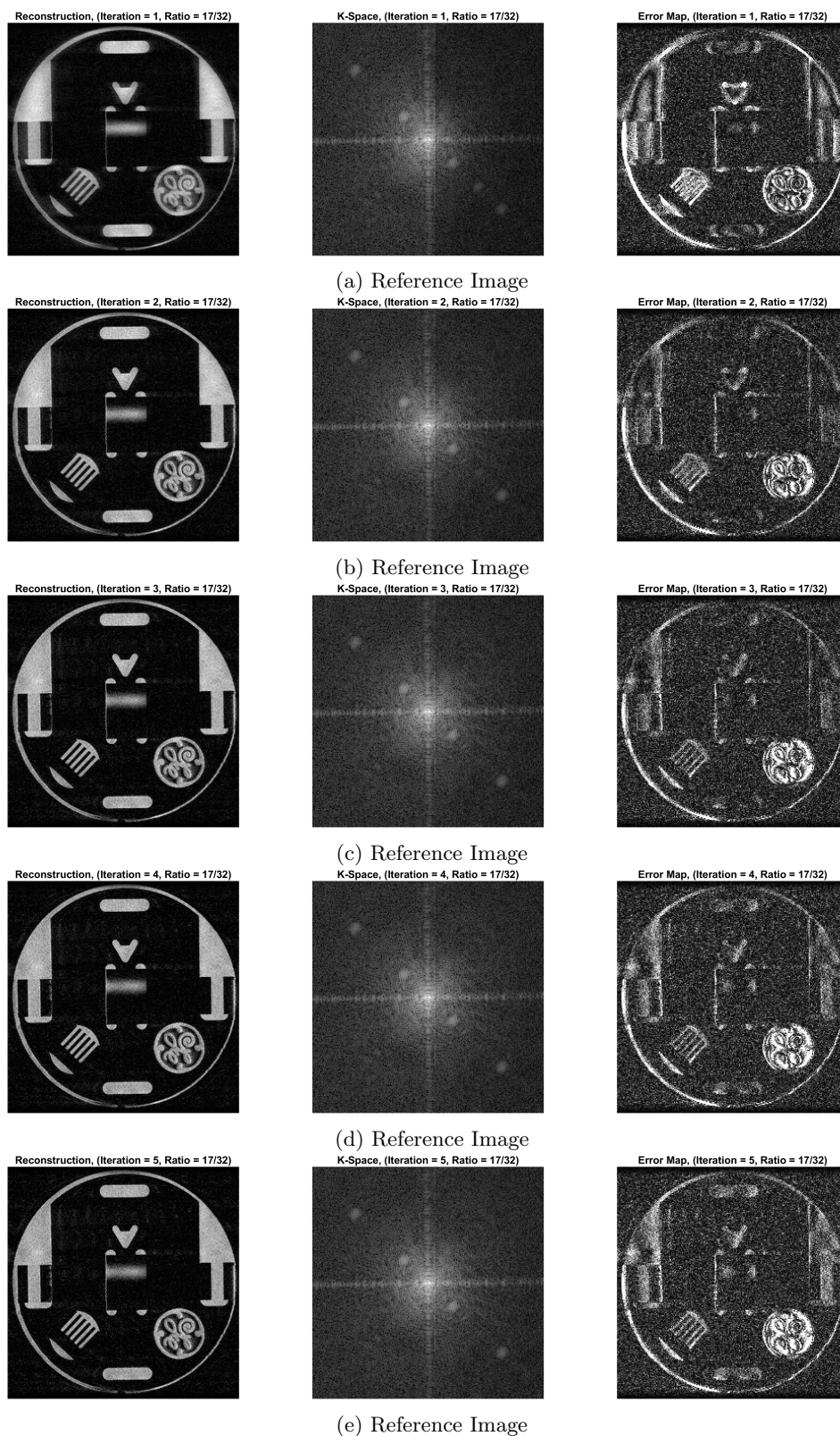


Figure B.2.1: First 5 Iterations of the POCS Algorithm with Ratio = 17/32

Extra Images for Question 4.5

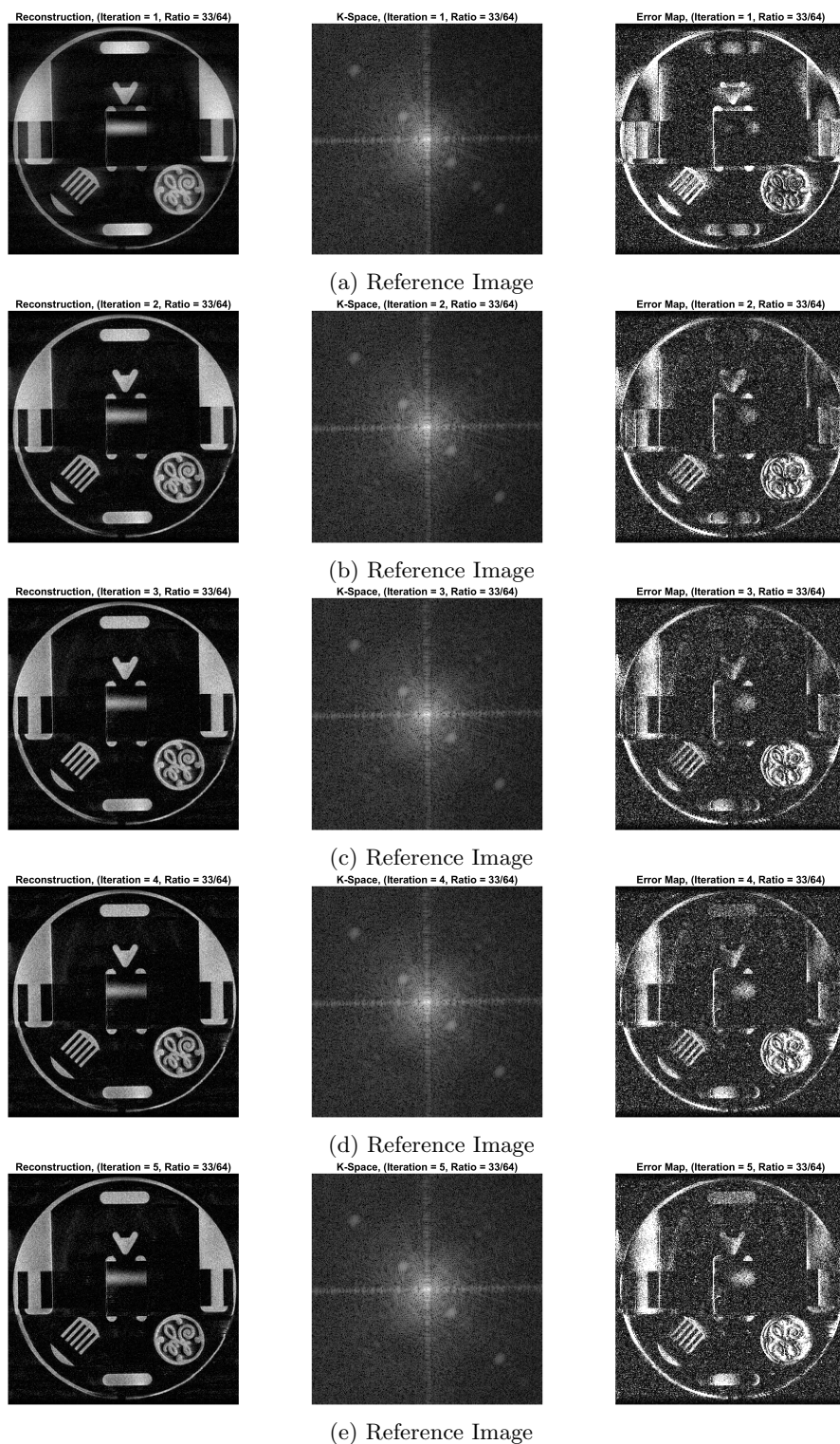


Figure B.3.1: First 5 Iterations of the POCS Algorithm with Ratio = 33/64

MATLAB Code

EEE 475 - Medical Imaging Reconstruction and Processing

Homework 1 - Tuna Şahin - 22201730

```
save = 1;
```

Question 1.1

```
mref = imread("arabam.png");
mref = im2double(rgb2gray(mref));
disp(size(mref))
imshow(mref,[])
title("Reference Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,"mref_q1.png",'Resolution',300)
end
mref_spectrum = fftshift(fft2(mref));
imshow(log(abs(mref_spectrum)+1),[])
title("Reference Image Spectrum")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,"mref_spectrum_q1.png",'Resolution',300)
end
```

Question 1.2, 1.3, 1.4:

```
for i = [2 4]
    mdown2 = mref(1:i:end,1:i:end);
    disp(size(mdown2))

    figure;
    imshow(mdown2,[])
    title(sprintf("Downsampled Image (Downsample = %d)", i))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    mdown2_spectrum = fftshift(fft2(mdown2));
    if save
        exportgraphics(gcf,strcat("mdown",num2str(i),"_q1.png"),'Resolution',300)
    end

    figure;
    imshow(log(abs(mdown2_spectrum)+1),[])
    title(sprintf("Spectrum of Downsampled Image (Downsample = %d)", i))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1.1, 1]);
    if save

exportgraphics(gcf,strcat("mdown",num2str(i),"_spectrum_q1.png"),'Resolution',300
)
    end
    shape = size(mref);
    padded_spectrum = zeros(shape);
    shape_down = size(mdown2_spectrum);
```

```

        padded_spectrum( ...
            floor((shape(1)-shape_down(1))/2)+1 : floor((shape(1)-shape_down(1))/
2)+shape_down(1), ...
            floor((shape(2)-shape_down(2))/2)+1 : floor((shape(2)-shape_down(2))/
2)+shape_down(2) ...
        ) = mdown2_spectrum;

figure;
imshow(log(abs(padded_spectrum)+1),[])
title(sprintf("Zero Padded Spectrum (Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save

exportgraphics(gcf,sprintf("zero_padded_spectrum_q1_downsample%d.png",i),'Resolut
ion',300)
end

mrecon2 = ifft2(ifftshift(padded_spectrum));
figure;
imshow(abs(mrecon2),[])
title(sprintf("Reconstruction of Zero Padded Image (Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1.2, 1.2]);
if save

exportgraphics(gcf,sprintf("recon_of_zero_padded_spectrum_q1_downsample%d.png",i)
,'Resolution',300)
end

mrecon2_n = normalise01(abs(mrecon2));
figure;
imshow(abs(mrecon2_n-mref),[])
title(sprintf("Error Magnitude Map(Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save

exportgraphics(gcf,sprintf("error_map_q1_downsample%d.png",i),'Resolution',300)
end
colorbar()
PSNR = psnr(mrecon2_n,mref)
SSIM = ssim(mrecon2_n,mref)
end

```

Question 1.5: No code required. Comments are provided on the report.

Question 2.1:

```

clf;
load("ge_phantom.mat") %loads variable kdata as 256x256 matrix
imshow(log(abs(kdata) + 1),[])
title("K Data, Provided")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save

```

```

    exportgraphics(gcf,"K Data, Provided.png",'Resolution',300)
end

mf = fftshift(fft2(fftshift(kdata)));
mref = abs(mf);
max_val = max(mref(:));
subplot(1,4,1)
imshow(mref,[])
title("Magnitude of the Image")
subplot(1,4,2)
imshow((angle(mf)),[])
title("Phase of the Image")
subplot(1,4,3)
imshow(real(mf),[])
title("Real Part of the Image")
subplot(1,4,4)
imshow(imag(mf),[])
title("Imaginary Part of the Image")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,"K Data, Provided, subplots.png",'Resolution',300)
end

```

Question 2.2:

```

clf;
mdown_pe = kdata(1:end,1:2:end);
imshow(log(abs(mdown_pe) + 1),[])
title("Downsampled K-Space (Downsample = 2)")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,"Downsampled K-Space (Downsample =
2).png",'Resolution',300)
end
mrecon = fftshift(fft2(fftshift(mdown_pe)));
imshow(abs(mrecon),[])
title("Reconstruction, PE Downsampling (Downsample = 2)")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,"Reconstruction, PE Downsampling (Downsample =
2).png",'Resolution',300)
end

```

Question 2.3, 2.4:

```

clf;

for i = [2 4]
    mdown_pe = kdata(1:end,1:i:end);
    mzero_pe = upsample(mdown_pe.',i).';

    figure;
    imshow(log(abs(mzero_pe) + 1),[])
end

```



```

title(sprintf("K-Space, Zero-Interleaving (Downsample = %d)", i))
mrecon_zero = fftshift(ifft2(ifftshift(mzero_pe)));
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,sprintf("K-Space, Zero-Interleaving (Downsample = %d).png", i), 'Resolution',300)
end

figure;
imshow(abs(mrecon_zero),[])
title(sprintf("Reconstruction, Zero-Interleaving (Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,sprintf("Reconstruction, Zero-Interleaving (Downsample = %d).png", i), 'Resolution',300)
end

figure;
imshow(abs(normalise01(abs(mrecon_zero))*max_val-normalise01(mref)*max_val), [0 0.2*max_val])
title(sprintf("Error Map (Downsample = %d)", i))
colorbar
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,sprintf("Reconstruction, Zero-Interleaving (Downsample = %d).png", i), 'Resolution',300)
end

PSNR = psnr(normalise01(abs(mrecon_zero)),mref)
SSIM = ssim(normalise01(abs(mrecon_zero)),mref)
end

```

Question 2.5, 2.6, 2.7 :

```

for i = [2 4]
    if i == 2
        kdata_cropped = kdata(1:end,65:end-64);
    elseif i == 4
        kdata_cropped = kdata(1:end,97:end-96);
    end
    figure;
    imshow(log(abs(kdata_cropped)+1),[])
    title(sprintf("K-Space, Cropped (Downsample = %d)", i))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    if save
        exportgraphics(gcf,sprintf("K-Space, Cropped (Downsample = %d).png", i), 'Resolution',300)
    end

    figure;
    imshow(abs(fftshift(ifft2(ifftshift(kdata_cropped))))),[])
    title(sprintf("Reconstruction, Cropped (Downsample = %d)", i))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
end

```

```

if save
    exportgraphics(gcf,sprintf("Reconstruction, Cropped (Downsample =
%d).png", i),'Resolution',300)
end

padded_kdata = zeros(size(kdata));
if i == 2
    padded_kdata(1:end,65:end-64) = kdata_cropped;
elseif i == 4
    padded_kdata(1:end,97:end-96) = kdata_cropped;
end
figure;
imshow(log(abs(padded_kdata)+1),[])
title(sprintf("K-Space, Cropped and Padded (Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,sprintf("K-Space, Cropped and Padded (Downsample =
%d).png", i),'Resolution',300)
end

mrecon_zp = abs(fftshift(ifft2(ifftshift(padded_kdata))));
figure;
imshow(mrecon_zp,[])
title(sprintf("Reconstruction, Cropped and Padded (Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,sprintf("Reconstruction, Cropped and Padded
(Downsample = %d).png", i),'Resolution',300)
end

figure;
imshow(abs(normalise01(mrecon_zp)*max_val-mref),[0 0.2*max_val])
title(sprintf("Error Map, Cropped and Padded (Downsample = %d)", i))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,sprintf("Error Map, Cropped and Padded (Downsample =
%d).png", i),'Resolution',300)
end

PSNR = psnr(mrecon_zp,mref)
SSIM = ssim(mrecon_zp,mref)
end

```

Question 2.8: No code required. Comments are provided on the report.

Question 3.1, 3.2, 3.3, 3.4,:

```

fractions = [1/8 1/16 1/32 1/64];
fracs = {"1/8","1/16","1/32","1/64"};
Npe = size(kdata,2);
for i = [1 2 3 4]

```

```

center = floor(Npe/2) + 1;
frac = fractions(i);

width = round(Npe * 2 * frac); % total lines kept
half = width/2;

cols = (center-half):(center+half-1);

kdata_center = zeros(size(kdata));
kdata_center(:, cols) = kdata(:, cols);

figure;
imshow(log(abs(kdata_center)+1),[])
title(strcat("K-Space Center, Cropped (Ratio = \pm", fracs{i},")"))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,strcat("K-Space Center, Cropped (Ratio = ",
num2str(i),").png"),'Resolution',300)
end

figure;
mrecon_center = fftshift(ifft2(ifftshift(kdata_center)));
imshow(angle(mrecon_center),[])
title(strcat("Phase of Reconstruction, Center (Ratio = \pm", fracs{i},")"))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,strcat("Phase of Reconstruction, Center (Ratio = ",
num2str(i),").png"),'Resolution',300)
end

figure;
p = exp(1j * angle(mrecon_center));
mpc = abs(real(mf.*conj(p)));
imshow(mpc,[])
title(strcat("Phase Corrected (Ratio = \pm", fracs{i},")"))
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,strcat("Phase Corrected (Ratio = ",
num2str(i),").png"),'Resolution',300)
end

mpcn = normalise01(mpc)*max_val;

figure;
imshow(abs(mref-mpcn),[0 0.2*max_val])
title(strcat("Error Map (Ratio = \pm", fracs{i},")"))
colorbar
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,strcat("Error Map (Ratio = ",
num2str(i),").png"),'Resolution',300)
end

PSNR = psnr(normalise01(mpcn),normalise01(mref))

```

```
SSIM = ssim(normalise01(mpcn),normalise01(mref))
end
```

Questions 3.5, 3.6: No code required. Comments are provided on the report.

Questions 4.1, 4.2, 4.3, 4.4, 4.5:

```
clf;
num_iter = 5;
Npe = size(kdata,2);
center = floor(Npe/2) + 1;
ratios = {"5/8","9/16","17/32","33/64"};
fractions = [1/8 1/16 1/32 1/64];
PSNRs = zeros(length(fractions),num_iter);
SSIMs = zeros(length(fractions),num_iter);
ZF_PSNRs = zeros(1,length(fractions));
ZF_SSIMs = zeros(1,length(fractions));
for i = [1 2 3 4]
    kdata_partial = zeros(size(kdata));

    if i == 1
        kdata_partial(1:end,1:end-96) = kdata(1:end,1:end-96);
    elseif i == 2
        kdata_partial(1:end,1:end-96-16) = kdata(1:end,1:end-96-16);
    elseif i == 3
        kdata_partial(1:end,1:end-96-16-8) = kdata(1:end,1:end-96-16-8);
    else
        kdata_partial(1:end,1:end-96-16-8-4) = kdata(1:end,1:end-96-16-8-4);
    end
    figure;
    imshow(log(abs(kdata_partial)+1),[])
    title(strcat("Partial K-Space, ZF (Ratio = ", ratios{i},")"))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    if save
        exportgraphics(gcf,strcat("Partial K-Space, ZF (Ratio = ",
num2str(i),").png"), 'Resolution',300)
    end

    figure;
    mrecon_zf = fftshift(iff2(iff2shift(kdata_partial)));
    imshow(abs(mrecon_zf),[])
    title(strcat("Reconstruction, Partial K-Space ZF (Ratio = ", ratios{i},")"))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    if save
        exportgraphics(gcf,strcat("Reconstruction, Partial K-Space ZF (Ratio =
", num2str(i),").png"), 'Resolution',300)
    end

    figure;
    imshow(abs(mref-normalise01(abs(mrecon_zf))*max_val),[0 0.2*max_val])
```

```

title(strcat("Error Map, Partial K-Space ZF (Ratio = ", ratios{i},"))
colorbar
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
if save
    exportgraphics(gcf,strcat("Error Map, Partial K-Space ZF (Ratio = ",
num2str(i),").png"),'Resolution',300)
end

SSIM = ssim(normalise01(abs(mrecon_zf)),normalise01(mref))
PSNR = psnr(normalise01(abs(mrecon_zf)),normalise01(mref))
ZF_SSIMs(i) = SSIM;
ZF_PSNRs(i) = PSNR;

fractions = [1/8 1/16 1/32 1/64];
frac = fractions(i);
width = round(Npe * 2 * frac); % total lines kept
half = width/2;
cols = (center-half):(center+half-1);
kdata_center = zeros(size(kdata));
kdata_center(:, cols) = kdata(:, cols);
mrecon_center = fftshift(ifft2(ifftshift(kdata_center)));
p = exp(1j * angle(mrecon_center));
kdata_loop = kdata_partial;
for iter = 1:num_iter
    mi = fftshift(ifft2(ifftshift(kdata_loop)));
    m_hat = abs(mi).*p;
    figure;
    subplot(1,3,1)
    imshow(abs(real(mi.*conj(p))),[])
    title(sprintf("Reconstruction, (Iteration = %d, Ratio = %s)", iter,
ratios{i}))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    if save
        exportgraphics(gcf,sprintf("Reconstruction, (Iteration = %d, Ratio =
%d).png", iter, fractions(i)), 'Resolution',300)
    end
    kdata_hat = fftshift(fft2(ifftshift(m_hat)));
    kdata_partial(:,center+half-1:end) = kdata_hat(:,center+half-1:end);
    kdata_loop = kdata_partial;
    subplot(1,3,2)
    imshow(log(abs(kdata_partial)+1),[])
    title(sprintf("K-Space, (Iteration = %d, Ratio = %s)", iter, ratios{i}))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    subplot(1,3,3)

    recon_show = abs(real(mi .* conj(p))); % your Q3-style phase
corrected magnitude
    err_show = abs(normalise01(recon_show) * max_val - mref);

    imshow(err_show, [0 0.2*max_val])
    title(sprintf("Error Map, (Iteration = %d, Ratio = %s)", iter,
ratios{i}))
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 1, 1]);
    if save

```

```

        exportgraphics(gcf,sprintf("K-Space, (Iteration = %d, Ratio =
%d).png", iter, fractions(i)),'Resolution',300)
    end
    PSNRs(i,iter) = psnr(abs(real(mi.*conj(p))),mref);
    SSIMs(i,iter) = ssim(abs(real(mi.*conj(p))),mref);
end
end
disp(PSNRs)
disp(SSIMs)

```

Questions 4.6, 4.7: No code required. But here is the code to graph the SSIM and PSNR values for question 4.

```

clf;
PSNRs = [ZF_PSNRs.' PSNRs]
SSIMs = [ZF_SSIMs.' SSIMs]

```

```

PSNRs = PSNRs(1:end,2:end)
SSIMs = SSIMs(1:end,2:end)

```

```

clf;
waterfall(flip(PSNRs))
xlabel("Iterations")
xticks(1:7)
xticklabels({"ZF", "1", "2", "3", "4", "5"})
ylabel("Ratio")
yticks([1 2 3 4])
yticklabels(flip(ratios))
zlabel("PSNR")
title("PSNR Values as a function of Number of Iterations and Ratios")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.3, 0.3]);
if save
    exportgraphics(gcf,"PSNR_graph.png",'Resolution',300)
end
waterfall(flip(SSIMs))
xlabel("Iterations")
xticks(1:7)
xticklabels({"ZF", "1", "2", "3", "4", "5"})
ylabel("Ratio")
yticks([1 2 3 4])
yticklabels(flip(ratios))
zlabel("SSIM")
title("SSIM Values as a function of Number of Iterations and Ratios")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.3, 0.3]);
if save
    exportgraphics(gcf,"SSIM_graph.png",'Resolution',300)
end

```

```

POCS_ssim = SSIMs(1:end,end);
POCS_psnr = PSNRs(1:end,end);

```

```

ZF_psnr = PSNRs(1:end,1);
ZF_ssim = SSIMs(1:end,1);
figure();
subplot(1,2,1)
fineplot([0:3]+0.1,POCS_ssim,"SSIM Values","Ratio","SSIM",[-0.3 3.5],[0.5
1],"on",[1600 300],"POCS","-o");
fineplot([0:3]+0.1,ZF_ssim,"SSIM Values","Ratio","SSIM",[-0.3 3.5],[0.5 1],"on",
[1600 300],"ZF","-o");
xticks([0:3]+0.1)
legend("Location","east")
xticklabels(ratios)
grid on
subplot(1,2,2)
fineplot([0:3]+0.1,POCS_psnr,"PSNR Values","Ratio","SSIM",[-0.3 3.5],[10
50],"on",[1600 300],"POCS","-o");
fineplot([0:3]+0.1,ZF_psnr,"PSNR Values","Ratio","SSIM",[-0.3 3.5],[10 50],"on",
[1600 300],"ZF","-o");
legend("Location","east")
xticks([0:3]+0.1)
xticklabels(ratios)
grid on

```

Functions

```

function x = normalise01(x)
    x = normalise(x - min(x(:)));
end

function x = normalise(x)
    x = x./max(x(:));
end

%x,y are the variables to be plotted.
%titlename,xaxisname,yaxisname,legendname are the labels in their respective
areas
%size is in pixels
%holdstate = 'on' is equivalent to "hold on"
%xlims and ylims are xlim and ylim
%style is the specifying string for color linestyle etc. like 'k--'
function plot_n =
fineplot(x,y,titlename,xaxisname,yaxisname,xlims,ylims,holdstate,size,legendN
ame,style)
    arguments
        x
        y
        titlename string = ''
        xaxisname string = ''
        yaxisname string = ''
        xlims (1,2) double = [min(x) max(x)]
        ylims (1,2) double = [min(y) max(y)]
        holdstate string = 'off'
        size (1,2) double = [400 400];
        legendName string = ''

```

```

        style = '- '
    end
    % x = [x(1) x];
    % y = [y(1) y];
    %even if the limits are reverse this section corrects the order
    if ylimits(1) > ylimits(2)
        ylimits = [ylimits(2) ylimits(1)];
    end
    if xlimits(1) > xlimits(2)
        xlimits = [xlimits(2) xlimits(1)];
    end

    %these measures are necessary for positioning arrows, labels etc.
    dx = ((xlimits(2)-xlimits(1))/65);
    dy = ((ylimits(2)-ylimits(1))/85);

    %this section is to not plot values that overlap with the arrows
    % lowindex = 1;
    % highindex = length(x);
    % for i = x
    %     if (i > (xlimits(1) + dx)) & (lowindex == 1)
    %         lowindex = find(x==i);
    %     end
    %     if (i > xlimits(2) - dx) & (highindex == 1)
    %         highindex = find(x==i)-1;
    %     end
    % end
    % if x(lowindex) > 0 %we allow points to be drawn at the origin
    %     lowindex = lowindex - 1;
    % end
    %
    % if x(highindex) < 0 %we allow points to be drawn at the origin
    %     highindex = highindex + 1;
    % end
    %
    % x = x(lowindex:highindex);
    % y = y(lowindex:highindex);

    %this section is responsible for axis configuration

    plot_n = plot(x,y,style,'displayName',legendName);
    xlim(xlimits);
    ylim(ylimits);
    set(gca,'XLabel','Visible','on');
    set(gca,'XAxisLocation','origin','box','off');
    set(gca,'YAxisLocation','origin');
    set(gca,'XAxis','FontWeight','bold');
    set(gca,'YAxis','FontWeight','bold');
    set(gca,'YLabel','Visible','on');
    set(gca,'Layer','top');
    set(gcf,'position',[0, 0, size(1) , size(2)]);

    %deletes the ticks that overlap with arrows

```



```

xticks('auto');
xt = xticks;
xticks(xt(2:length(xt)-1));
yticks('auto');
yt = yticks;
yticks(yt(2:length(yt)-1));

% determining the ylevel to draw the arrows on the x axis
if ((ylimits(1) * ylimits(2)) < 0)
    xal = 0;
elseif ylimits(1) < 0
    xal = ylimits(2);
else
    xal = ylimits(1);
end

%plotting the arrows
hold on
if xlimits(2) > 0
    plot([(xlimits(2)-dx) xlimits(2) (xlimits(2)-dx)], [xal+dy xal xal-
dy], 'k', 'HandleVisibility', 'off') %xaxis right arrow
end
if xlimits(1) < 0
    plot([xlimits(1)+dx xlimits(1) xlimits(1)+dx], [xal+dy xal xal-
dy], 'k', 'HandleVisibility', 'off') %xaxis left arrow
end
plot([0 0], [ylimits(2) ylimits(1)], 'k', 'HandleVisibility', 'off') % y axis
if ylimits(2) > 0
    plot([-dx/2 0 dx/2], [ylimits(2)-dy ylimits(2) ylimits(2)-
dy], 'k', 'HandleVisibility', 'off') %yaxis top arrow
end
if ylimits(1) < 0
    plot([-dx/2 0 dx/2], [ylimits(1)+dy ylimits(1)
ylimits(1)+dy], 'k', 'HandleVisibility', 'off') %yaxis bottom arrow
end

%repositioning title & label locations
label_h1 = xlabel(xaxisname);
label_h1.Position(1) = xlimits(2)+dx; % change horizontal position of xlabel.
label_h1.Position(2) = dy; % change vertical position of xlabel.c5
label_h2 = ylabel(yaxisname, rotation=0);
label_h2.Position(1) = dx; % change horizontal position of ylabel.
label_h2.Position(2) = ylimits(2)+dy; % change vertical position of ylabel.
title(titlename);
% if ((xlimits(1) * xlimits(2)) < 0) & (xlimits(1)+xlimits(2) < ((xlimits(2)-
xlimits(1))/3))
%     set(get(gca, 'title'), 'Position', [20*dx ylimits(2)-dy]) %prevents the
title from colliding with ylabel
% end

%holds or doesnt hold
if strcmp(holdstate, 'off')
    hold off
else

```

```
        hold on  
    end  
end
```